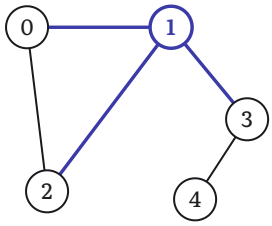
Обходы графов: DFS и BFS

Хранение графа, обход в глубину и в ширину, типовые задачи. Графы неориентированные, без петель и кратных рёбер; об ориентированных — в конце §2.

Обозначения. $G = (V, E)$ — граф, $n = |V|$, $m = |E|$. Вершины нумеруются $0, \dots, n-1$; если во входных данных нумерация с 1, при чтении вычитаем 1. $\deg v$ — степень вершины v, $\sum_v \deg v = 2m$: каждое ребро учтено в степенях обоих концов.

1. Хранение графа

1.1. Три способа



(а) граф

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	0
2	1	1	0	0	0
3	0	1	0	0	1
4	0	0	0	1	0

(б) матрица смежности

```
(0, 1)
(0, 2)
(1, 2)
(1, 3)
(3, 4)
```

(в) список рёбер

```
0: 1, 2
1: 0, 2, 3
2: 0, 1
3: 1, 4
4: 3
```

(г) списки смежности

Рис. 1. Один граф, три способа хранения. Выделено, где хранятся соседи вершины 1.

Матрица смежности. Двумерный массив g: $g_{uv} = 1$, если $\{u, v\} \in E$, иначе 0. Для неориентированного графа матрица симметрична. Проверка ребра — $O(1)$, перебор соседей вершины — $O(n)$ (просмотр строки), память — $O(n^2)$.

Список рёбер. Массив из m пар (u, v), память — $O(m)$. Чтобы найти соседей вершины, нужно просмотреть все рёбра. Удобен, когда рёбра нужны как отдельные объекты, например чтобы отсортировать их по весу.

Списки смежности. Для каждой вершины v — массив $adj[v]$ её соседей. Ребро $\{u, v\}$ записывается дважды: v в $adj[u]$ и u в $adj[v]$. Память — $O(n + m)$, перебор соседей v — $O(\deg v)$.

	Матрица	Список рёбер	Списки смежности
Память	$O(n^2)$	$O(m)$	$O(n + m)$
Есть ли ребро $\{u, v\}$	$O(1)$	$O(m)$	$O(\deg u)$
Перебор соседей v	$O(n)$	$O(m)$	$O(\deg v)$
Полный обход (DFS, BFS)	$O(n^2)$	$O(nm)$	$O(n + m)$

Для обходов обычно используют списки смежности. Матрица оправдана, когда граф плотный: при m порядка n^2 оценка $O(n^2)$ совпадает с $O(n + m)$, а ребро вдобавок проверяется за $O(1)$ — это удобно, например, при работе с дополнением графа. По памяти матрица возможна при n порядка 10^3; при $n = 2 \cdot 10^5$ в ней $4 \cdot 10^{10}$ ячеек.

Листинг 1. Чтение графа.

C++

```cpp
vector<vector<int>> adj;  // глобальный

// в main:
int n, m;
cin >> n >> m;
adj.assign(n, {});
for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    u--, v--;  // нумерация с 0
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

PYTHON

```python
import sys
input = sys.stdin.readline

n, m = map(int, input().split())
adj = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1; v -= 1  # нумерация с 0
    adj[u].append(v)
    adj[v].append(u)
```

1.2. Неявный граф: клетчатое поле

Поле $h \times w$ задано строками из символов «.» (свободная клетка) и «#» (стена). Вершины графа — свободные клетки, рёбра соединяют свободные клетки с общей стороной.

Такой граф не хранят: соседей клетки (r, c) получают сдвигами $(\pm 1, 0)$ и $(0, \pm 1)$ с проверкой границ поля и стен (листинг 2). Вершин не больше hw, рёбер не больше $2hw$, поэтому обход работает за $O(hw)$. Если клетке нужен один номер, берут $r \cdot w + c$.

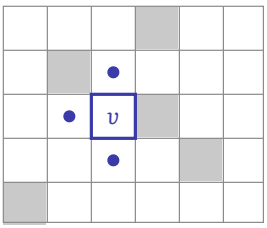


Рис. 2. Соседи клетки v; справа стена.

Листинг 2. Чтение поля и перебор соседей клетки (r, c).

C++

```cpp
int h, w;
cin >> h >> w;
vector<string> g(h);
for (auto& row : g) cin >> row;

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};
// соседи клетки (r, c)
for (int k = 0; k < 4; k++) {
    int nr = r + dr[k], nc = c + dc[k];
    if (nr < 0 || nr >= h) continue;
    if (nc < 0 || nc >= w) continue;
    if (g[nr][nc] == '#') continue;
    // (nr, nc) — сосед
}
```

PYTHON

```python
h, w = map(int, input().split())
g = [input().strip() for _ in range(h)]

D = [(-1, 0), (1, 0), (0, -1), (0, 1)]
# соседи клетки (r, c)
for dr, dc in D:
    nr, nc = r + dr, c + dc
    if not (0 <= nr < h and 0 <= nc < w):
        continue
    if g[nr][nc] == '#':
        continue
    # (nr, nc) — сосед
```

2. Обходы

2.1. Обход в глубину (DFS)

Из вершины v DFS по очереди переходит в каждого ещё не посещённого соседа и рекурсивно продолжает обход из него; вернувшись, переходит к следующему соседу. Массив `used` не даёт зайти в вершину второй раз, в `parent[u]` запоминается вершина, из которой пришли в u. Чтобы обойти весь граф, `dfs(s)` запускают из каждой ещё не посещённой вершины s.

Листинг 3. Обход в глубину.

C++

```cpp
// adj — из листинга 1
vector<int> used, parent;

void dfs(int v) {
    used[v] = 1;
    for (int u : adj[v])
        if (!used[u]) {
            parent[u] = v;
            dfs(u);
        }
}

// в main, после чтения графа:
used.assign(n, 0);
parent.assign(n, -1);
for (int s = 0; s < n; s++)
    if (!used[s]) dfs(s);
```

PYTHON

```python
import sys
sys.setrecursionlimit(10**6)

def dfs(v):
    used[v] = True
    for u in adj[v]:
        if not used[u]:
            parent[u] = v
            dfs(u)

used = [False] * n
parent = [-1] * n
for s in range(n):
    if not used[s]:
        dfs(s)
```

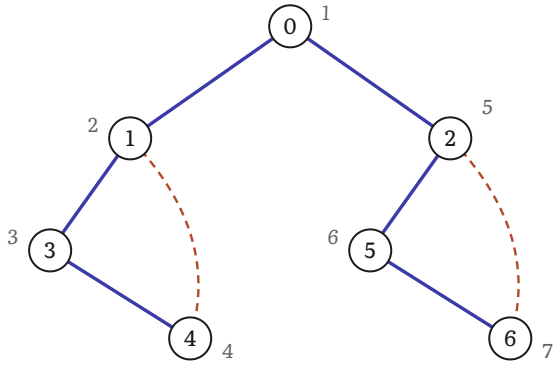


Рис. 3. DFS из вершины 0, соседи перебираются по возрастанию номера. Сплошные рёбра — дерево DFS, пунктир — обратные рёбра 4–1 и 6–2. Числа у вершин — порядок входа.

Свойства DFS.

1. Вызов `dfs(s)` посещает ровно вершины, достижимые из s, то есть компоненту связности s.
2. Каждая вершина обрабатывается один раз, и её список смежности просматривается один раз. Суммарная длина списков равна $\sum_v \deg v = 2m$, отсюда время $O(n+m)$. Дополнительная память — $O(n)$, глубина рекурсии — до n.
3. Рёбра $\{parent[v], v\}$ образуют *дерево* DFS — остовное дерево компоненты; для всего графа получается остовный лес.
4. Любое ребро соединяет вершину с её предком или потомком в дереве DFS; рёбра вне дерева называют *обратными*. Пусть $\{u, v\}$ — ребро и v посещена раньше u. В момент входа в v вершина u ещё не посещена. До выхода из v DFS просмотрит всех соседей v, в том числе u, поэтому u будет посещена внутри вызова `dfs(v)`, то есть окажется в поддереве v.
5. Для любого t первые t посещённых вершин образуют связное множество: родитель каждой из них, кроме стартовой, посещён раньше неё.

Обход со стеком. `sys.setrecursionlimit` поднимает только программный лимит. Упадёт ли глубокая рекурсия, зависит от версии интерпретатора, поэтому на графах с длинными путями в Python надёжнее обход со стеком (листинг 4): вершина помечается в момент добавления в стек.

Листинг 4. Обход со стеком (Python).

PYTHON

```python
def traverse(s):
    used[s] = True
    stack = [s]
    while stack:
        v = stack.pop()
        for u in adj[v]:
            if not used[u]:
                used[u] = True
                parent[u] = v
                stack.append(u)
```

Это не DFS: порядок посещения другой, и ребро вне дерева может соединять разные ветви. В треугольнике 0–1–2 из вершины 0 в стек сразу попадают 1 и 2, и ребро 1–2 соединяет две ветви. Свойства 1–3 и 5 выполняются, свойство 4 — нет. Поэтому такой обход годится, когда важно только, какие вершины достижимы: компоненты, поиск цикла, двудольность.

Нерекурсивный DFS хранит в стеке пары «вершина, индекс следующего соседа»; в этом пособии он не нужен.

2.2. Обход в ширину (BFS)

BFS обрабатывает вершины в порядке возрастания расстояния от s: сначала s, затем все вершины на расстоянии 1, затем на расстоянии 2 и т. д. Вершины хранятся в очереди. Вершина получает `dist` в момент добавления в очередь, поэтому попадает в очередь один раз.

Листинг 5. Обход в ширину из вершины s.

C++

```cpp
vector<int> dist(n, -1), parent(n, -1);
queue<int> q;
dist[s] = 0;
q.push(s);
while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v])
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            parent[u] = v;
            q.push(u);
        }
}
```

PYTHON

```python
from collections import deque

dist = [-1] * n
parent = [-1] * n
dist[s] = 0
q = deque([s])
while q:
    v = q.popleft()
    for u in adj[v]:
        if dist[u] == -1:
            dist[u] = dist[v] + 1
            parent[u] = v
            q.append(u)
```

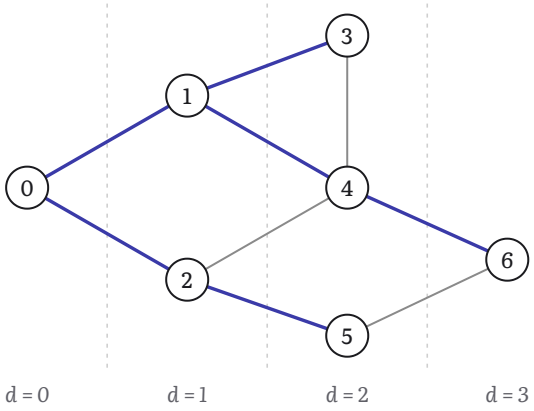


Рис. 4. BFS из вершины 0. Порядок извлечения из очереди: 0, 1, 2, 3, 4, 5, 6. Выделены рёбра дерева кратчайших путей; остальные рёбра (2–4, 3–4, 5–6) соединяют вершины одного или соседних слоёв.

Свойства BFS.

1. Посещаются вершины компоненты s. Каждая вершина добавляется в очередь один раз, и её список смежности просматривается один раз: время $O(n+m)$, память $O(n)$.
2. Вершины извлекаются в порядке неубывания `dist`; в каждый момент в очереди лежат вершины не более чем двух слоёв, d и $d+1$.
3. После обхода $dist[v]$ — длина кратчайшего пути из s в v в рёбрах; для недостижимых вершин остаётся -1.
4. Для любого ребра $\{u, v\}$ компоненты $|dist[u] - dist[v]| \le 1$: ребро соединяет вершины одного слоя или соседних.
5. Рёбра $\{parent[v], v\}$ образуют *дерево кратчайших путей*: путь по `parent` из v в s кратчайший.

Ориентированный граф. Ребро $u \to v$ кладут только в $adj[u]$: в листинге 1 остаётся одна строка `push_back` (`append`). DFS и BFS работают без изменений, и $dist[v]$ — длина кратчайшего пути из s в v вдоль направления рёбер. Свойство 4 у обоих обходов в ориентированном графе не выполняется.

3. Типовые задачи

3.1. Компоненты связности

Задача. Найти число компонент связности и для каждой вершины — номер её компоненты.

Решение. Перебираем вершины по порядку. Каждая ещё не помеченная вершина начинает новую компоненту: из неё запускается обход, который помечает всю компоненту её номером (свойство 1 DFS). Каждая вершина помечается один раз за весь алгоритм, а её список просматривается один раз (свойство 2 DFS), поэтому суммарное время — $O(n+m)$, а не n раз по $O(n+m)$.

Листинг 6. Компоненты связности.

C++

```cpp
vector<int> comp;

void dfs(int v, int c) {
    comp[v] = c;
    for (int u : adj[v])
        if (comp[u] == -1)
            dfs(u, c);
}

// в main:
comp.assign(n, -1);
int k = 0;  // число компонент
for (int v = 0; v < n; v++)
    if (comp[v] == -1)
        dfs(v, k++);
```

PYTHON

```python
comp = [-1] * n
k = 0  # число компонент
for s in range(n):
    if comp[s] != -1:
        continue
    comp[s] = k
    stack = [s]
    while stack:
        v = stack.pop()
        for u in adj[v]:
            if comp[u] == -1:
                comp[u] = k
                stack.append(u)
    k += 1
```

Чтобы граф с k компонентами стал связным, нужно добавить минимум $k-1$ рёбер: каждое новое ребро уменьшает число компонент не больше чем на 1, а $k-1$ рёбер, соединяющих первую компоненту с каждой из остальных, достаточно.

3.2. Поиск цикла

Задача. Определить, есть ли в графе цикл.

Решение. Рёбра $\{parent[v], v\}$ образуют остовный лес (свойство 3 DFS). Граф без циклов — это и есть лес, поэтому цикл есть тогда и только тогда, когда какое-то ребро в этот лес не попало. Обход встречает такое ребро как уже посещённого соседа, отличного от родителя. Рассуждение не зависит от порядка обхода, поэтому годится и для обхода со стеком.

Листинг 7. Проверка на наличие цикла.

C++

```cpp
vector<int> used;
bool cycle = false;

void dfs(int v, int p) {
    used[v] = 1;
    for (int u : adj[v]) {
        if (u == p) continue;
        if (used[u]) cycle = true;
        else dfs(u, v);
    }
}

// в main:
used.assign(n, 0);
for (int s = 0; s < n; s++)
    if (!used[s]) dfs(s, -1);
```

PYTHON

```python
used = [False] * n
parent = [-1] * n
cycle = False
for s in range(n):
    if used[s]:
        continue
    used[s] = True
    stack = [s]
    while stack:
        v = stack.pop()
        for u in adj[v]:
            if u == parent[v]:
                continue
            if used[u]:
                cycle = True
            else:
                used[u] = True
                parent[u] = v
                stack.append(u)
```

Чтобы вывести сам цикл, нужен рекурсивный DFS: при первом таком обнаружении u — предок v (свойство 4 DFS), и цикл — это путь от v вверх по `parent` до u и ребро $\{u, v\}$.

Через число рёбер. Лес из k деревьев на n вершинах содержит ровно $n - k$ рёбер, где k находим обходом из §3.1. Поэтому цикл есть $\Leftrightarrow m > n - k$, а граф является деревом $\Leftrightarrow$ он связен и $m = n - 1$.

3.3. Обход дерева

Задача. Дано дерево с корнем $root$. Найти глубину каждой вершины — число рёбер на пути от корня.

Решение. В дереве у вершины v есть рёбра только к родителю и к детям, а дети к моменту обработки v ещё не посещены: путь от корня к ним проходит через v. Поэтому массив `used` не нужен, достаточно передавать родителя p и не возвращаться в него. Так же вниз по дереву передают любую величину, которая зависит от пути от корня, например сумму весов на нём. Время $O(n)$.

Листинг 8. Глубины вершин дерева.

C++

```cpp
vector<int> depth;

// used не нужен: в родителя p не идём
void dfs(int v, int p, int d) {
    depth[v] = d;
    for (int u : adj[v])
        if (u != p)
            dfs(u, v, d + 1);
}

// в main:
depth.assign(n, 0);
dfs(root, -1, 0);
```

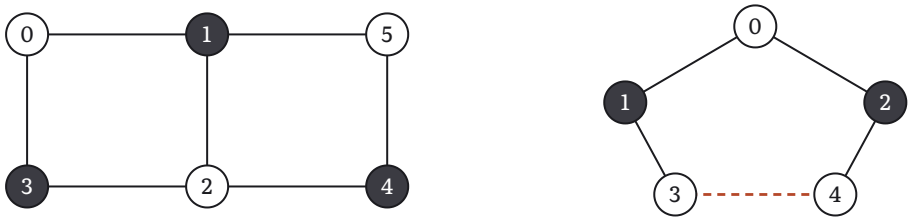
PYTHON

```python
depth = [0] * n
stack = [(root, -1, 0)]
while stack:
    v, p, d = stack.pop()
    depth[v] = d
    for u in adj[v]:
        if u != p:
            stack.append((u, v, d + 1))
```

3.4. Двудольность

Задача. Проверить, можно ли раскрасить вершины в два цвета так, чтобы концы каждого ребра были разного цвета.

Решение. Граф двудолен тогда и только тогда, когда в нём нет цикла нечётной длины. Красим каждую компоненту обходом: стартовая вершина получает цвет 0, сосед вершины цвета c — цвет $1 - c$. Цвет каждой вершины однозначно определяется цветом стартовой, поэтому найденное ребро с одноцветными концами означает, что раскраски нет. При обходе BFS цвет вершины равен $dist[v] \bmod 2$, и по свойству 4 BFS конфликт возможен только на ребре внутри слоя (рис. 5б).



(а) двудольный граф *(б) цикл длины 5*

Рис. 5. Цвета 0 и 1 — светлые и тёмные вершины. (б) BFS из 0: слой 1 — вершины 1, 2, слой 2 — вершины 3, 4; ребро 3–4 внутри слоя соединяет вершины одного цвета.

Листинг 9. Проверка на двудольность обходом в ширину.

C++

```cpp
vector<int> color(n, -1);
bool ok = true;
for (int s = 0; s < n; s++) {
    if (color[s] != -1) continue;
    color[s] = 0;
    queue<int> q;
    q.push(s);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u : adj[v]) {
            if (color[u] == -1) {
                color[u] = 1 - color[v];
                q.push(u);
            } else if (color[u] == color[v])
                ok = false;
        }
    }
}
```

PYTHON

```python
from collections import deque

color = [-1] * n
ok = True
for s in range(n):
    if color[s] != -1:
        continue
    color[s] = 0
    q = deque([s])
    while q:
        v = q.popleft()
        for u in adj[v]:
            if color[u] == -1:
                color[u] = 1 - color[v]
                q.append(u)
            elif color[u] == color[v]:
                ok = False
```

3.5. Кратчайший путь

Задача. Найти путь из s в t с наименьшим числом рёбер.

Решение. BFS из s (листинг 5) даёт длину $dist[t]$ (свойство 3 BFS). Сам путь восстанавливается по массиву `parent` от t к s и разворачивается (свойство 5 BFS). Время $O(n + m)$.

Листинг 10. Восстановление пути после BFS.

C++

```cpp
// после BFS из s
vector<int> path;
if (dist[t] != -1) {
    for (int v = t; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
}
// path = s, ..., t
```

PYTHON

```python
# после BFS из s
path = []
if dist[t] != -1:
    v = t
    while v != -1:
        path.append(v)
        v = parent[v]
    path.reverse()
# path = s, ..., t
```

3.6. Расстояние до ближайшего из нескольких источников

Задача. На клетчатом поле отмечены клетки-источники. Для каждой клетки найти расстояние до ближайшего источника.

Решение. Все источники кладутся в очередь сразу с расстоянием 0, затем выполняется обычный BFS. Это равносильно BFS из фиктивной вершины, соединённой со всеми источниками: расстояние от неё минус 1 и есть расстояние до ближайшего источника (свойство 3 BFS). Один обход вместо k: $O(hw)$ вместо $O(k \cdot hw)$.

0	1	2	3		6	5	4
1			4		5		3
2	3	4	5	5	4		2
	4			4	3	2	1
6	5	6		3	2	1	**0**

Рис. 6. Поле 5×8, источники в углах (выделены). В клетке — расстояние до ближайшего источника; серые клетки — стены.

Листинг 11. BFS из нескольких источников на поле; поле, `dr`, `dc` и `D` — из листинга 2.

C++

```cpp
vector d(h, vector<int>(w, -1));
queue<pair<int, int>> q;
for (auto [r, c] : sources) {
    d[r][c] = 0;
    q.push({r, c});
}
while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();
    for (int k = 0; k < 4; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= h) continue;
        if (nc < 0 || nc >= w) continue;
        if (g[nr][nc] == '#') continue;
        if (d[nr][nc] != -1) continue;
        d[nr][nc] = d[r][c] + 1;
        q.push({nr, nc});
    }
}
```

PYTHON

```python
from collections import deque

d = [[-1] * w for _ in range(h)]
q = deque()
for r, c in sources:
    d[r][c] = 0
    q.append((r, c))
while q:
    r, c = q.popleft()
    for dr, dc in D:
        nr, nc = r + dr, c + dc
        if not (0 <= nr < h and 0 <= nc < w):
            continue
        if g[nr][nc] == '#':
            continue
        if d[nr][nc] != -1:
            continue
        d[nr][nc] = d[r][c] + 1
        q.append((nr, nc))
```

4. Типичные ошибки

- **Нумерация с 1.** Вершины во входе часто нумеруются с 1: вычитайте 1 при чтении или заводите массивы размера $n + 1$.
- **Несвязный граф.** Обход из одной вершины посещает только её компоненту. Для всего графа нужен внешний цикл по вершинам, как в листингах 3, 6, 7 и 9.
- **Пометка в BFS.** Вершину помечают при добавлении в очередь. Если помечать при извлечении, одна вершина может попасть в очередь по разу от каждого соседа и обрабатываться несколько раз.
- **Переполнение `int`.** Количества рёбер, пар вершин и суммы по компонентам легко превышают $2^{31} - 1 \approx 2{,}1 \cdot 10^9$: например, пар вершин при $n = 10^5$ около $5 \cdot 10^9$. В C++ нужен `long long`.
- **Мультитест.** Если тестов много и ограничена сумма n, массивы заводят и очищают на размер текущего теста, а не на максимальный.
- **Рекурсия в Python.** Глубина DFS доходит до n, например на пути из 10^5 вершин. Используйте обход со стеком.
- **Строки поля в Python.** `sys.stdin.readline` оставляет `'\n'` в конце строки. Без `strip()` ширина поля окажется на 1 больше, а `'\n'` сойдёт за свободную клетку.

5. Задачи

Задачи с Codeforces, примерно по возрастанию сложности.

1. 1829E. The Lakes
2. 217A. Ice Skating
3. 862B. Mahmoud and Ehab and the bipartiteness
4. 520B. Two Buttons
5. 580C. Kefa and Park
6. 329B. Biridian Forest
7. 377A. Maze
8. 601A. The Two Routes
9. 659E. New Reform
10. 1093D. Beautiful Graph